



عملي

3



8



1200 sp

كلية الهندسة المعلوماتية

السنة الثانية

توجيهات المجمع و التعليمات الشرطية

م. عبير ميا



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 28/04/2026

بنیان الحاسوب ا

تذكرة

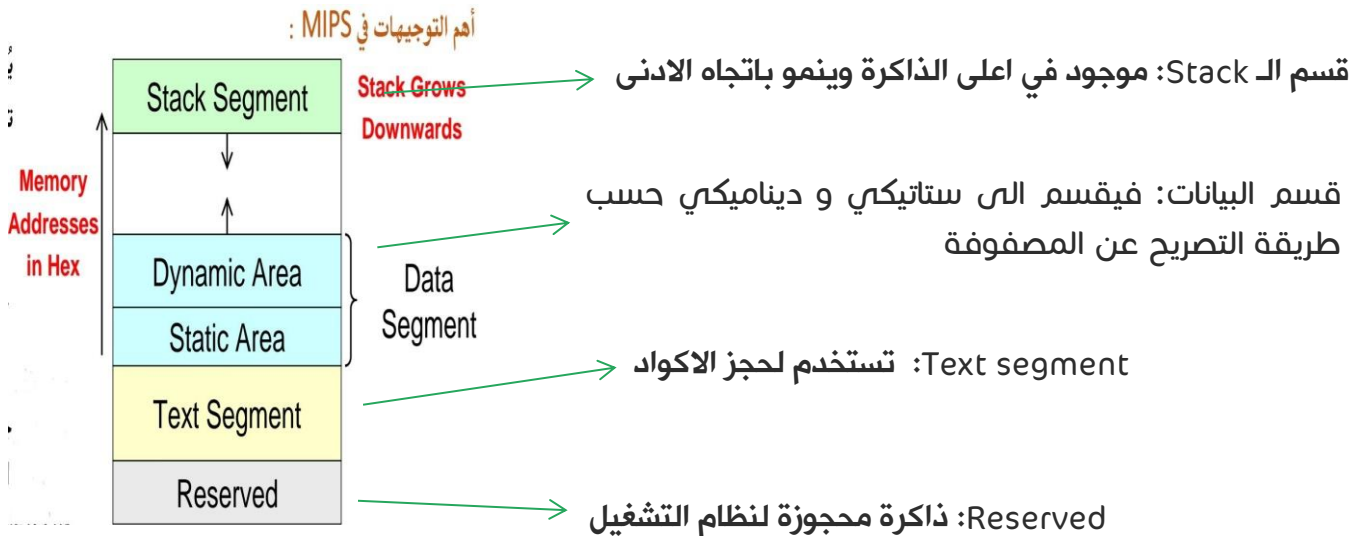
- السجل هو عبارة عن وحدة تخزين داخلية ضمن المعالج تتميز بسرعة هائلة تستخدم للاحتفاظ بالبيانات التي يتم العمل عليها حاليا والتي يتم الوصول اليها بشكل متكرر وهذا يعني ان المعالج لا يمكنه اجراء العمليات الحسابية الا على البيانات الموجودة داخل السجلات فقط وليس مباشرة من الذاكرة RAM.
- إذا كان المجمع يحول من تعليمات زائفة الى تعليمات حقيقية يستخدم السجل \$at.

عناوين المحاضرة الثالثة

-عندما نصرح عن مصفوفة في لغات عالية المستوى كيف تصبح في لغة المجمع؟
-التعليمات الشرطية

في المحاضرة السابقة كنا نتعامل مع عناوين متفرقة في الذاكرة حيث تفيد في الهندسة العكسية عندما نريد الوصول لعنوان معين سواء عمل تخريبي او إصلاحي أما في هذه المحاضرة سيكون لدينا برنامج بلغة عالية المستوى وما يقابله بلغة التجميع.

مخطط تمثيل للذاكرة:



أهم التوجيهات في لغة التجميع

.data: تستخدم للإشارة الى بداية قسم البيانات حيث يتم تعريف المصفوفات التي تخزن في الذاكرة.

.text: تستخدم للإشارة الى بداية قسم التعليمات التنفيذية (البرامج).

توضع المتغيرات المعرفة في قسم .data .

تخزن الدوال والعمليات الحسابية والاستدعاءات عند الترجمة في قسم .text .

التوجيهات التصريحية لحجز المساحات داخل قسم البيانات:

.word: تعني احجز مكان في الذاكرة يتسع لكلمة واحدة وفي معالجنا هنا يحجز 32 بت، و يوجد ثلاث طرق لاستعمالها:

1. .word 7 X: و تعني قم بتعريف متغير x قيمته الابتدائية 7، و يقابلها في لغة C: `int x = 7;`

2. .word 1, 2, 3, 4 a: اما الشكل الثاني مصفوفة من الاعداد الصحيحة مع القيم ويقابلها في لغة C: `int a[4]={1,2,3,4};`

3. .word 0:5 arr: أما الشكل الثالث يحجز في الذاكرة مصفوفة من 5 كلمات القيمة الابتدائية لها اصفار ويقابلها في لغة C: `int arr[5]={0,0,0,0,0};`

.space: تستخدم لحجز مساحة فارغة بعدة بايتات، حيث نقوم باختيار اسم وعدد مثلاً 20 يعبر عن عدد البايتات كما يلي:

size1: .space20

حيث يقابلها في لغة السي: `int size1[5];`

.byte: لحجز قيمة لمحرّف واحد، مثل: `Var1: .byte 'A'`

.ascii: سلسلة محارف (String) لا تنتهي ب '\0'، مثل: `str1: .ascii "HI"`

.asciiz: سلسلة محارف (String) منتهية ب '\0'، مثل: `str2: .asciiz "null terminated string"`

.align n: لضمان محاذاة البيانات في الذاكرة على عناوين تقبل القسمة على 2^n .

.float: لاسناد قيمة عشرية (بها فاصلة)



فائدة أريسيزية:

تفيد كتابة .align n في تصحيح المحاذاة عند استعمال .ascii أو .asciiz لتخزين strings أو عند استعمال .byte لتخزين بايتات أو أحرف، وذلك لأنه مثلاً في حالة قيامنا بتخزين بايت واحد أو اثنين ستتم محاذاة العنصر التالي الذي يتم تخزينه مباشرة بعد العنصر الأول (أي سيبدأ من البايت الثالث للكلمة نفسها و ليس من بداية كلمة جديدة) و هذا سيسبب خطأ محاذاة عند محاولة الوصول إليه مما يؤدي إلى خطأ في برنامجنا، فلمعالجة هذه الحالات نلجأ إلى استعمال .align n لجعل المحاذاة صحيحة مع 2^n و هذا ما يعيدنا إلى المحاذاة الصحيحة. (مع العلم أنك قد لا تصادف هذا الخطأ لأن المحاكيات مثل MARS تعالجه تلقائياً)

double-: لاسناد قيمة عشرية و لكن مع خانات أكثر بعد الفاصلة (مثل لغات البرمجة التي تجمع نوع المتحولات double).

ملاحظة: string (سنتوسع أكثر عنها في المحاضرة القادمة)

التعليمة الزائفة la (load address):

حيث تستخدم لتحميل موقع عنوان في الذاكرة الى سجل (العنوان مؤلف من 32 بت):
 .data :
 arr: .word 1,2,3,4
 .text:
 La \$s1 , arr
 حيث نقوم بتحميل موقع عنوان في الذاكرة الى سجل (العنوان مؤلف من 32 بت):
 حيث نحجز مصفوفة فيها 4 عناصر من نوع word
 تقوم التعليمة la بشحن قيمة السجل \$s1 بعنوان المصفوفة arr.
 حيث يقوم المجمع بتحويلها الى التعليمتين الحقيقيتين المكافئتين لها : ori , lui كما سبق.

Bkpt	Address	Code	Basic	Source
	0x00400000	0x3c011001	lui \$1, 0x00000001	5: la \$s1, arr
	0x00400004	0x34310000	ori \$17, \$1, 0x00000000	
	0x00400008	0x24100001	addiu \$16, \$0, 0x00000001	6: li \$s0, 1

قمنا هنا بإزالة الاصفار على اليسار لان التعليمات lui , ori , addiu تعليمات حقيقية لا تتسع لأكثر من 16 بت

تمرين:

اكتب برنامج بلغة التجميع يقوم بحجز مصفوفة مؤلفة من ثلاثة اعداد 4 8 0 ثم إجراء شحن للسجل \$t0 بعنوان بداية المصفوفة ثم إجراء or بين اول عنصرين في المصفوفة وتخزين الناتج في الموقع الثالث من المصفوفة مع توضيح القيم النهائية للذاكرة والسجلات.

الحل:

أولا قمنا بحجز 3 كلمات فنقوم بوضعهم في قسم البيانات لانها تقوم بتعريف المصفوفات

```
.data
Numbers: .word 4, 8, 0
.text
la $t0, Numbers
lw $t1, 0($t0)
lw $t2, 4($t0)
or $t1, $t1, $t2
sw $t1, 8($t0)
```

التي تتخزن في الذاكرة، واسم المصفوفة اختياري.

ثم قمنا بشحن السجل ببداية المصفوفة.

حملنا العنصر الاول والثاني من الذاكرة الى السجل ومن العنصر الأول الى الثاني مقدار الازاحة 4.

هنا نريد تخزين الناتج في الموقع الثالث اي الانزياح 8.

لتوضيح القيم النهائية للذاكرة والسجلات:

قبل التنفيذ في الذاكرة كانت القيم 4 8 0 في المواقع 0x10010008 , 0x10010004 , 0x10010000 على الترتيب
 أما بعد التنفيذ فأصبحت قيم هذه العناوين 4 8 12 على الترتيب (الأرقام ممثلة بالنظام العشري) ، أما بالنسبة للسجلات

Coprocc 1		Coprocc 0	
Registers			
Name	Number	Value	
\$zero	0	0x00000000	
\$at	1	0x10010000	
\$v0	2	0x00000000	
\$v1	3	0x00000000	
\$a0	4	0x00000000	
\$a1	5	0x00000000	
\$a2	6	0x00000000	
\$a3	7	0x00000000	
\$t0	8	0x10010000	
\$t1	9	0x0000000c	
\$t2	10	0x00000008	
\$t3	11	0x00000000	
\$t4	12	0x00000000	

فقد أصبحت قيم السجلين \$t1 و \$t2 هي 0x0000000c و 0x00000008 على الترتيب (الأرقام ممثلة بالنظام الست عشري) وذلك بسبب عملية الـ or بين \$t1 و \$t2 الذي كانا في البداية 4 و 8،

أما السجل \$at فقد أصبح يحمل القيمة 0x10010000 وذلك بسبب استعمالنا للتعليمية الزائفة la حيث قام المجمع باستخدامه كوسيط أثناء تحويلها إلى المكافئين الحقيقيين ori , lui.

التعليمات الشرطية BEQ, BNE

- التعليمة beq rs,rt,L1

أي تقارن بين rs ,rt (إذا كانا متساويان)

في حال تمت المساواة اذهب لـ L1 (عبارة عن قيمة عددية)، والا عند عدم تحقق الشرط تابع بشكل تسلسلي

- التعليمة bne rs,rt,L1

في حالة عدم المساواة بين rs ,rt اذهب لـ L1 والا اكمل بشكل تسلسلي.

■ ملاحظة: beq , bne تعليمات تتفرع شرطيا (Branch if Equal) & (Branch if Not Equal)

- تعليمة القفز غير المشروط J : J L1

وهي من النمط J-type وتعني اقفز لـ L1 بدون أي شروط



اقفز لأحلامك بلا شروط كتعليمة J ...

و لا تأبه بالعوائق كما beq ...



مثال: اكتب مكافئ العبارة الشرطية التالية بلغة التجميع MIPS بفرض: $x=\$s0, y=\$s1, z=\$s2$ ما هي قيمة x عند انتهاء التنفيذ (بفرض القيم الابتدائية: $x=-5, y=5$ ثم بحالة $y=6$)

High-level code «C»

“If / Else Statement”

«MIPS assembly code»?

```
int x = -5, y = 5;
int d;
d = x + y;
if (d == 0)
    x = 2;
else
    x=10;
```

```
add $t0, $s0, $s1    # t0 = x + y
bne $t0, $zero, else # اقفز إن كان المجموع لا يساوي صفر
li $s0, 2            # x = 2 (تنفذ فقط إذا كان المجموع صفراً)
j end
else: li $s0, 10    # x= 10 (تنفذ فقط إذا كان لا يساوي صفراً)
end:
```

```
add $t0, $s0, $s1
beq $t0, $zero, do
li $s0, 10
j end
do: li $s0, 2
end:
```

$x = \$s0 = -5, y = \$s1 = 5 \Rightarrow$

$\$s0 = 2$

$x = \$s0 = -5, y = \$s1 = 6 \Rightarrow$

$\$s0 = 10$

\$s0	16	2
\$s1	17	5

\$s0	16	10
\$s1	17	6

مثال:

اكتب في برنامج المحاكاة MARS برنامجاً بلغة التجميع MIPS يقابل ما يلي :

```
if (i == j)
    f = f + k;
    g = h - 1;
else f = f - h;
    g = - k;
```

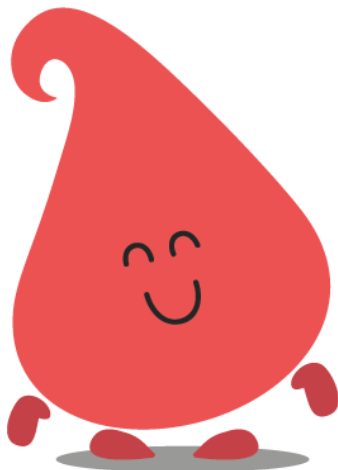
بفرض أن المجتوى الابتدائي للسجلات:

$i : \$t0=4, j : \$t1=6, h : \$t2=8, k : \$t3=1, g : \$v1=12, f : \$v0 =9$

؟ ثم وضع ما هو المجتوى النهائي لكل من السجلين $\$v0, \$v1$ ؟

القيمة النهائية للسجلات ستكون : $\$v0 = 1$

$\$v1 = -1$



```
addi $t0, $zero, 4 # $t0 is i
addi $t1, $zero, 6 # $t1 is j
addi $t2, $zero, 8 # $t2 is h
addi $t3, $zero, 1 # $t3 is k
addi $v0, $zero, 9 # $v0 is f
addi $v1, $zero, 12 # $v1 is g
```

إذا تحقق الشرط أي كان
 \$t0 = \$t1 السجل \$t0, \$t1, L1 #
 اقفز إلى القسم l1
 ولكن إن لم يتحقق الشرط تابع
 للسطر التالي
 j L2

```
L1:
add $v0, $v0, $t3
addi $v1, $t2, -1
لكي نمنع تنفيذ أي تعليمة
بعد نقفز # j End
لقسم فارغ و هو End
هذا القسم لجزء الـ else
L2:
sub $v0, $v0, $t2
sub $v1, $zero, $t3
End:
```

أسئلة و تمارين:

السؤال الأول:

لدينا مصفوفة A تحتوي على 4 عناصر عشوائية ومصفوفة B تحتوي على 3 عناصر فارغة

والمطلوب

اكتب برنامج بلغة MIPS Assembly باستخدام التعليمات المناسبة يقوم بما يلي:

تحميل العنصر الأول والثاني من المصفوفة A ثم مقارنة القيمتين إذا كانتا متساويتين خزن قيمة العنصر الأول في الموقع الأخير من المصفوفة B والا احسب مجموعهما وخزن الناتج في الموقع الثاني من المصفوفة B

الحل:

```
.data
A: .word 1, 2, 3, 4

B: .space 12
.text
l1: add $t1, $t1, $t2
la $t0, A

la $s0, B
lw $t2, 4($t0)
lw $t3, 8($t0)
```

```
lw $t4, 12($t0)
bne $t1, $t2, l1
sw $t1, 8($s0)
sw $t1, 4($s0)
End:
lw $t1, 0($t0)
```

السؤال الثاني:

اكتب برنامج بلغة التجميع MIPS في برنامج المحاكاة MARS يقوم بما يلي :

1. التصريح عن مصفوفتين A مكونة من 3 عناصر وقيمها عشوائية ومصفوفة B مكونة من 3 عناصر فارغة
2. تحميل العنصرين الأول والثاني من المصفوفة A واجراء عملية and بينهما اذا كانت القيمة الناتجة تساوي الصفر
خزن العنصر A[2] في الموقع الأول من المصفوفة الثانية B والا اوجد ناتج and بين القيمة السابقة والعنصر الثالث من المصفوفة A وخزن النتيجة بالموقع الثالث من المصفوفة B

الحل:

```
.data
A: .word 2,4,8
B: .space 12
.text
la $t0 ,A
la $s0 ,B
lw $t1,0($t0)
lw $t2,4($t0)
lw $t3,8($t0)
and $t4,$t1,$t2
bne $t4,$0,L1
sw $t3,0($s0)
j end
L1:
and $t5,$t3,$t4
sw $t5,8($s0)
end:
```

السؤال الثالث:

اكتب برنامج بلغة التجميع MIPS في برنامج المحاكاة MARS يقوم بما يلي :

- التصريح عن مصفوفة A مكونة من 3 عناصر قيمها عشوائية ومصفوفة B مكونة من 3 عناصر فارغة
اذا كان A[0] يساوي الصفر خزن في B[0] القيمة 1 والا خزن في المصفوفة B معكوس عناصر المصفوفة A

الحل:

```
.data
A: .word 0,2,3
B: .space 12
.text
la $t0,A
lw $s0,0($t0)
lw $s1,4($t0)
lw $s2,8($t0)
la $s4,B
beq $s0,$zero,L1
j L2
L1:
li $s5,1
sw $s5,0($s4)
j end
L2:
sub $s5,$zero,$s0
sub $s6,$zero,$s1
sub $s7,$zero,$s2
sw $s5,0($s4)
sw $s6,4($s4)
sw $s7,8($s4)
end:
```


أسئلة للتدريب

أسئلة نظرية

1 - ما السبب الرئيسي الذي يجعل المعالج ينفذ العمليات الحسابية على البيانات الموجودة في السجلات بدلاً من RAM تنفيذها مباشرةً على الذاكرة ؟

(a) السجلات تمتلك سعة تخزين أكبر من RAM

(b) ذاكرة الـ RAM مخصصة حصرياً لنظام التشغيل ولا يمكن استخدامها في العمليات الحسابية

(c) السجلات هي وحدات تخزين داخلية موجودة داخل المعالج وتوفر سرعة عالية جداً

(d) ذاكرة الـ RAM تخزن التعليمات فقط بينما السجلات تخزن جميع المتغيرات

الإجابة : c

2 - أيّ من العبارات التالية يصف بشكل صحيح استخدام المقطعين `.data` و `.text` في لغة التجميع MIPS ؟

(a) المقطع `.text` يُستخدم لتخزين المصفوفات، بينما المقطع `.data` يخزن البرنامج التنفيذي.

(b) يمكن استخدام `.data` و `.text` بالتبادل حسب حجم البرنامج.

(c) المقطع `.data` يُستخدم لتعريف المصفوفات والمتغيرات في الذاكرة، بينما المقطع `.text` يحدّد بداية التعليمات القابلة للتنفيذ.

(d) المقطع `.data` ينمو نزولاً من أعلى الذاكرة لمنع حدوث تجاوز في السعة (Overflow).

الإجابة : c

3 - ما الفرق الوظيفي بين التوجيهين `.ascii` و `.asciiz` عند تعريف السلاسل النصية (Strings) في الذاكرة ؟

(a) `.ascii` يُستخدم للمحارف المفردة فقط، بينما `.asciiz` يُستخدم للجمل الكاملة.

(b) `.asciiz` يضيف تلقائياً محرف النهاية Null (القيمة 0) إلى نهاية السلسلة النصية، بينما `.ascii` لا يفعل ذلك.

(c) `.asciiz` يُستخدم للأعداد العشرية (Floating Point)، بينما `.ascii` يُستخدم للنصوص.

(d) `.ascii` يغيّر حجمه ديناميكياً أثناء التنفيذ، بينما `.asciiz` ثابت.

الإجابة : b

4 - لماذا يُعد التوجيه `align` مفيداً عند تخزين البايتات المفردة أو المحارف في الذاكرة؟

(a) يضمن محاذاة البيانات على عناوين ذاكرة تقبل القسمة على 2، مما يمنع أخطاء المحاذاة عند الوصول إلى الكلمات (Words) اللاحقة.

(b) يقوم تلقائياً بترتيب عناصر المصفوفة تصاعدياً في الخلفية.

(c) يضغط السلسلة النصية لتوفير مساحة في مكدس الذاكرة (Stack).

(d) يضمن تحويل جميع أحرف السلسلة إلى أحرف كبيرة (Uppercase).

الإجابة : a

5 - تُعتبر التعليمة `la` (Load Address) تعليمة وهمية (Pseudo-instruction) إلى أي تعليمتين حقيقيتين بطول 16 بت يقوم المجمع (Assembler) عادةً بتحويلها خلف الكواليس؟

lw , sw (a)

beq , bne (b)

lui , ori (c)

add , sub (d)

الإجابة : c

كتابة اكواد

السؤال الأول:

أنشئ مصفوفة `Arr1` تحتوي على 3 قيم عشوائية، ومصفوفة `Arr2` فارغة بحجم 12 بايت .

نفذ عملية OR منطقية على مستوى البتات (`or`) بين العنصر الأول والعنصر الثاني من `Arr1`

إذا كانت النتيجة تساوي صفراً، خزّن العنصر الثالث من `Arr1` في الموقع الأول من `Arr2`

إذا لم تكن النتيجة تساوي صفراً، نفّذ عملية AND منطقية على مستوى البتات (`and`) بين ناتج عملية الـ OR السابقة

والعنصر الثالث من `Arr1`، ثم خزّن الناتج النهائي في الموقع الثاني من `Arr2`

Answer:

```
.data
a: .word 4 , 3 , 3 , 19 #random numbers
b: .space 16
.text
#loading the addresses
la $t0 , a
la $t1 , b
```

```
#load the second element
lw $s0 , 4($t0)
#load the third element
lw $s1 , 8($t0)
beq $s0 , $s1 , equal #check whether they are equal
#if they are not equal it will continue sequentially
sub $v0 , $s0 , $s1 #we compute the difference
sw $v0 , 0($t1) #and store it in the first position of b
j end #so that it doesn't continue sequentially and executes
what's inside the (equal) label
```

```
equal: # if we are here then the 2nd and 3rd elements are
equal
lw $s2 , 12($t0) #we load the fourth element of a
sw $s2 , 4($t1) # and then store it in the last (second)
position in b
end:
```

السؤال الثاني:

عرّف مصفوفة X تحتوي على 3 قيم عشوائية، ومصفوفة فارغة Y مخصصة لاحتواء 3 عناصر .

تحقق مما إذا كان العنصر الأول في X يساوي 5

إذا كان يساوي 5، خزّن القيمة 1 في جميع عناصر Y الثلاثة .

إذا لم يكن يساوي 5، اطرح العنصر الأول من العنصر الثاني في X، ثم خزّن الناتج في الموقع الأول من Y

Answer:

```
.data
Arr1 : .word 2 , 4 , 19 #random numbers
Arr2: .space 12
.text
#loading the addresses
la $t0 , Arr1
la $t1 , Arr2

lw $s0 , 0($t0) #loading the first element of Arr1
lw $s1 , 4($t0) #loading the second element of Arr1
lw $s2 , 8($t0) #loading the third element of Arr1

or $v0 , $s0 , $s1 #performed a logical OR operation and
stored the value in $v0
beq $v0 , $zero , equal
#if it doesn't equal zero

and $v1 , $s2 , $v0 #performed a logical AND operation
```

#between the value of the OR operation and the 3rd element of Arr1

sw \$v1 , 4(\$t1) #stored the result of the AND operation in the 2nd position of Arr2

j end

equal:

#if it does equal zero

sw \$s2 , 0(\$t1) #stored the 3rd element of Arr1 in the 1st position of Arr2

end:

السؤال الثالث:

عرّف مصفوفة X تحتوي على 3 قيم عشوائية، ومصفوفة فارغة Y مخصّصة لاحتواء 3 عناصر .

تحقّق مما إذا كان العنصر الأول في X يساوي 5

إذا كان يساوي 5، خزّن القيمة 1 في جميع عناصر Y الثلاثة .

إذا لم يكن يساوي 5، اطرح العنصر الأول من العنصر الثاني في X، ثم خزّن الناتج في الموقع الأول من Y

Answer:

.data

x: .word 5 , 5 , 1 #three random numbers

y: .space 12

.text

#loading the addresses

la \$t0 , x

la \$t1 , y

#putting the value 5 in a register to compare it with the 1st element of x

addi \$a0 , \$zero , 5

lw \$s0 , 0(\$t0) #loading the 1st element of x

beq \$s0 , \$a0 , equal

#if they are not equal

lw \$s1 , 4(\$t0)

sub \$v0 , \$s1 , \$s0 #subtracting the first element from the second

sw \$v0 , 0(\$t1) #storing the value in the 1st position of y

j end

equal:

#if they are equal

#having a register with the value of 1 to store it in y

```
addi $a1 , $zero , 1

#storing 1 in every position in y
sw $a1 , 0($t1)
sw $a1 , 4($t1)
sw $a1 , 8($t1)

end:
```

"The person you are becoming is shaped by the choices you make when no one is watching.

Every small act of discipline today is a vote for the future you want tomorrow."

